TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos

Profesores:

Ing. Jorge Arun Zambrano Cedeño. Mg.

Titulaciones	Semestre
• *Tecnologías de la Información en Línea*	*Quinto*

NOMBRES Y APELLIDOS:

TERAN JINEZ GABRIEL ALEJANDRO

PARALELO:

"A"

SEMESTRE:

QUINTO

ACTIVIDAD:

Estudio de Caso - Sistema de Control de Rescate y Adopción Animal

PERIODO SEPTIEMBRE 2025 - ENERO 2026

Actividades a desarrollar en la Unidad 1

Actividad # 1: Estudio de caso - Sistema de Control de Rescate y Adopción Animal

OBJETIVO GENERAL:

Diseñar (simulación de programa real) la arquitectura lógica de un sistema de gestión de rescate y adopción animal que permita aplicar correctamente patrones de diseño de software y flujos de trabajo profesionales (Gitflow), fortaleciendo la capacidad de análisis, abstracción y buenas prácticas en el desarrollo de soluciones escalables, reutilizables y mantenibles.

OBJETIVOS ESPECIFICOS:

- Identificar los actores principales y sus funciones, y establecer los requerimientos funcionales y no funcionales del sistema.
- Diseñar la estructura lógica de la solución mediante diagramas de casos de uso, clases y secuencia.
- Aplicar los patrones Singleton, Observer y Factory Method para resolver necesidades específicas del sistema.
- Organizar la arquitectura utilizando el patrón MVC para separar la interfaz, la lógica de negocio y el acceso a los datos.
- Establecer una estrategia de control de versiones con Git y GitHub utilizando ramas organizadas bajo el modelo Gitflow.
- Promover la escalabilidad, mantenibilidad, reutilización de componentes y trazabilidad de los cambios realizados.

➢ **ANALISIS DEL PROBLEMA**

PLANTEAMIENTO.

Se plantea desarrollar una aplicación web con una arquitectura organizada en módulos. Los usuarios podrán acceder según sus funciones y permisos.

El sistema quedará estructurado de la siguiente manera (tentativamente):

1. Registro del reporte de rescate.
2. Identificación y clasificación del animal.
3. Creación del expediente según la especie.

4. Asignación de un voluntario o veterinario.
5. Seguimiento del estado del caso.
6. Registro de tratamientos y observaciones médicas.
7. Preparación y gestión del proceso de adopción.
8. Generación de notificaciones y consulta del historial.

Los estados iniciales del caso serán: rescatado, en tratamiento, disponible para adopción, en proceso de adopción y adoptado. También se contempla el estado cerrado, para casos que deban finalizar por razones justificadas, como una derivación o la conclusión administrativa del expediente.

Las transiciones estarán controladas por reglas de negocio. Por ejemplo, un animal no podrá pasar a estar disponible para adopción mientras tenga un tratamiento pendiente que requiera atención veterinaria.

IDENTIFICACIÓN DE ACTORES

Los principales actores y sus funciones son:

ACTOR	FUNCIÓN
Ciudadano o reportante	Reportar el rescate, proporciona la ubicación, describe la situación.
Voluntario veterinario	Registrar observaciones y tratamientos, dar seguimiento a los animales bajo su responsabilidad, Actualizar información
Administrador del refugio	Gestionar y asignar voluntarios para cada reporte de rescate, supervisar dichos casos, controlar y supervisar los casos y administrar el proceso de adopción

El proceso seria, el ciudadano reporta el caso (información inicial) estado, ubicación situación entre otros. El Administrador registra el caso, asigna voluntarios y da seguimiento al caso. El Voluntario registra observaciones, sigue tratamientos y da seguimiento a los animales a su cargo.

REQUISITOS DEL SISTEMA

REQUISITOS FUNCIONALES:

Tentativamente los requisitos funcionales describen las acciones que el sistema debe permitir realizar.

No	Requisito	Descripción
01	Registrar el rescate	Permite crear un nuevo caso con fechas de ingreso, ubicación y datos del ciudadano reportante
02	Clasificar especie	Identifica si el animal corresponde a un animal doméstico (perro, gato u otros) o fauna silvestre
03	Crear expediente	Genera un expediente especifico según la especie del animal
04	Asignar responsables	Permite que el administrador asigne un voluntario veterinario a un caso
05	Gestionar estados	Actualiza el estado del caso conforme a las etapas del rescate, diagnóstico y tratamiento
06	Gestionar adopción	Registra solicitudes, revisar requisitos y formalizar la adopción
07	Consultar expedientes	Busca casos y revisa la información autorizada de cada animal
08	Gestionar usuarios	Permite al administrador controlar las cuentas y funciones de acceso

REQUISITOS NO FUNCIONALES

No	Requisito	Descripción
01	Escalabilidad	Permitir incorporar nuevos módulos, especies y funcionalidades sin rediseñar completamente el sistema
02	Mantenibilidad	Mantener el código organizado en clases y módulos con responsabilidades definidas.
03	Trazabilidad	Registrar los cambios relevantes de los casos y conservar el historial de versiones del código en Git.
04	Seguridad	Controlar el acceso según roles y proteger la información de usuarios y expedientes.

05	Disponibilidad	Facilitar el acceso al sistema durante las actividades habituales del refugio, con procedimientos de respaldo y recuperación
06	Integridad	Validar los datos y evitar transiciones de estado incompatibles con las reglas del proceso.
07	Reutilización	Diseñar componentes que puedan utilizarse en distintos módulos sin duplicar innecesariamente el código

➢ DISEÑO DEL SISTEMA

El sistema tendrá una arquitectura por capas que utilizará MVC como capa de organización de la aplicación. Además de:

Capa de presentación: Contiene las vistas y los formularios utilizados por ciudadanos, voluntarios-veterinariosy administradores.

Capa de control: Recibe las solicitudes de los usuarios, valida los datos de entrada y dirige las operaciones hacia los servicios correspondientes.

Capa de lógica de negocio: Administra las reglas relacionadas con los rescates, expedientes, asignaciones, tratamientos y adopciones. En esta capa se aplican los patrones Singleton, Observer y Factory Method.

Capa de acceso a datos: Contiene los repositorios encargados de almacenar y consultar la información en la base de datos.

Módulo de notificaciones: Informa a los usuarios responsables sobre nuevos casos y cambios importantes. (En esta propuesta, las notificaciones pueden registrarse en la plataforma y, posteriormente, ampliarse a correo electrónico u otros canales).

PATRONES DE DISEÑO

Se utilizarán seis patrones Singleton, observer, factory method, MVC, builder y state, los cuales nos ayudarán a cubrir la creación de objetos como la comunicación entre módulos y la organización estructural de la app.

Patron	Función dentro del sistema	Problema que resuelve
Singleton	Proporciona una instancia compartida del servicio central cuando sea necesario.	Evita que distintos módulos creen instancias independientes de un mismo coordinador (reduce duplicidad)
Observer	Notifica eventos a voluntarios y veterinarios interesados la creación de casos, las asignaciones y otros cambios	Evita que el servicio de rescates tenga que conocer todas las clases
Factory Method	Crea el tipo de expediente correspondiente a cada especie (perros, gatos, fauna silvestre)	Evita concentrar toda la lógica de creación de expedientes en un único bloque
MVC	Separa la presentación, el control y el modelo de la aplicación	Evita mezclar el código de las pantallas con las reglas de negocio y el acceso a los datos, haciendo más difícil el mantenimiento.
Builder	Construye expedientes complejos paso a paso. Agrega información de forma ordenada y comprobada	Evita que la información de cada caso se dispersa y desordenada
State	Controla el comportamiento y las transiciones de los estados del rescate y la adopción.	Evita concentrar todas las reglas de funcionamiento en múltiples condiciones dentro de una sola clase.

Relación entre los patrones y los principios SOLID

Los patrones se complementan con los principios SOLID para mantener un diseño modular.

PRINCIPIO	APLICACIÓN EN EL SISTEMA
Responsabilidad única (SRP)	Cada clase se concentra en una función, como registrar casos, crear expedientes o notificar eventos.
Abierto/cerrado (OCP)	Se pueden incorporar nuevos tipos de expediente o estados mediante nuevas implementaciones, sin modificar innecesariamente los componentes existentes.
Sustitución de Liskov (LSP)	Las implementaciones de Expediente y EstadoCaso deben respetar los contratos definidos por sus abstracciones.
Segregación de interfaces (ISP)	Se definen interfaces específicas para repositorios, notificaciones y estados.
Inversión de dependencias (DIP)	Los servicios dependen de abstracciones como CasoRepository y Notificador, en lugar de depender directamente de implementaciones concretas.

Integración de los patrones en la arquitectura

Los patrones no funcionan como módulos aislados, sino como mecanismos que colaboran en el flujo de gestión del caso. Cuando se registra un animal, Factory Method permite seleccionar el tipo de expediente apropiado. Builder incorpora los datos del animal, sus antecedentes, tratamientos y observaciones.

Una vez creado el caso, State controla la etapa en la que se encuentra y las transiciones permitidas. Observer comunica los eventos relevantes a los suscriptores. Singleton puede proporcionar una instancia compartida del servicio central de gestión de casos, cuando sea necesario. MVC organiza el acceso a estas funcionalidades desde la interfaz y los controladores.

Diagrama de la arquitectura general

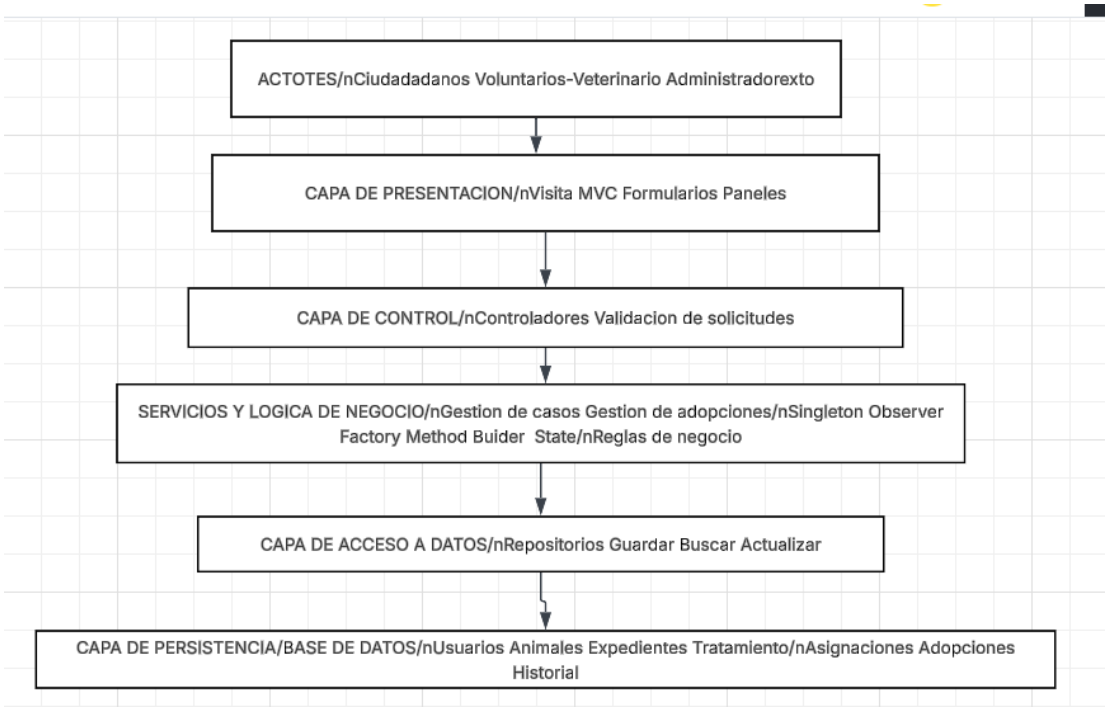


Diagrama de Clase

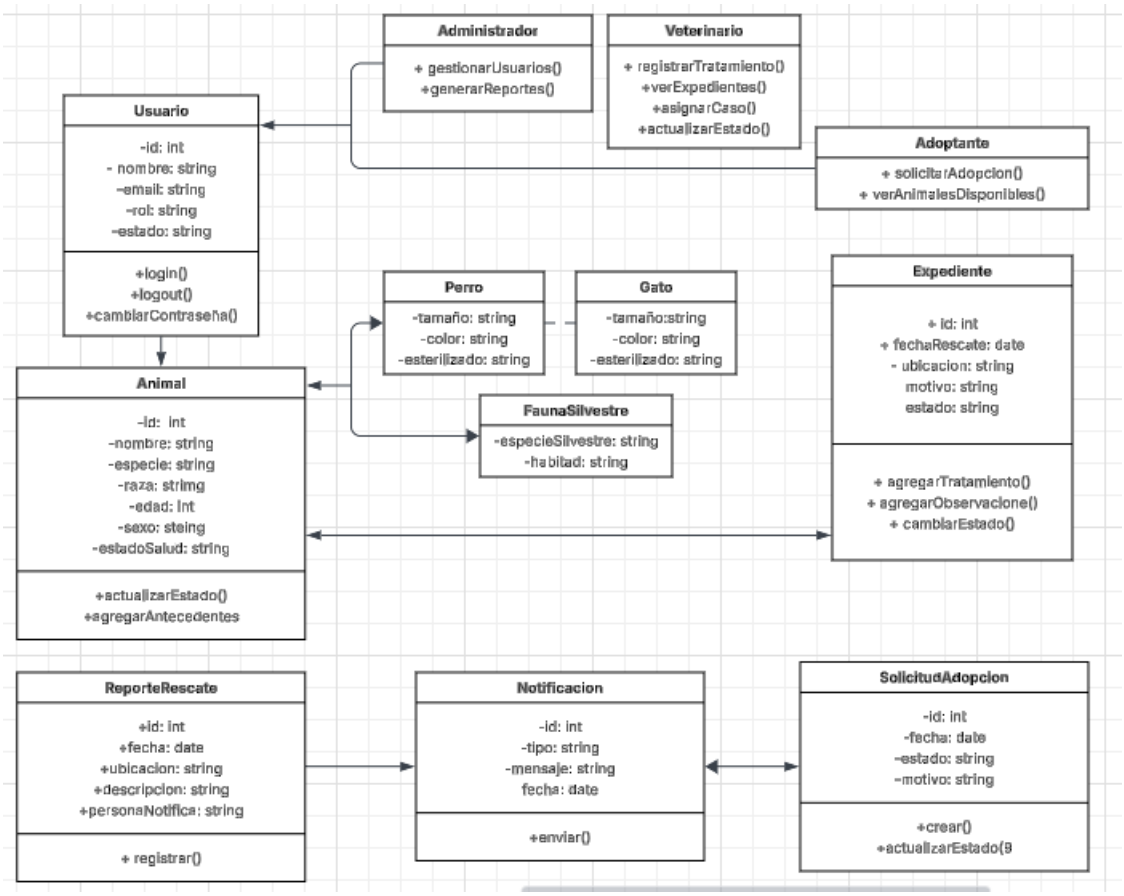
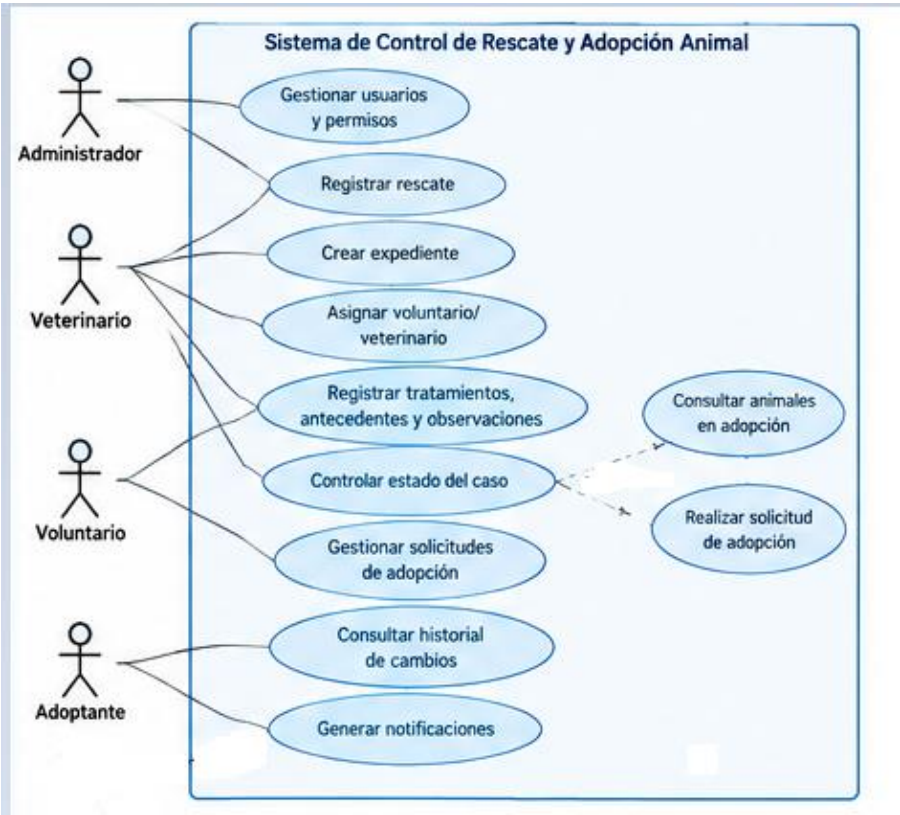
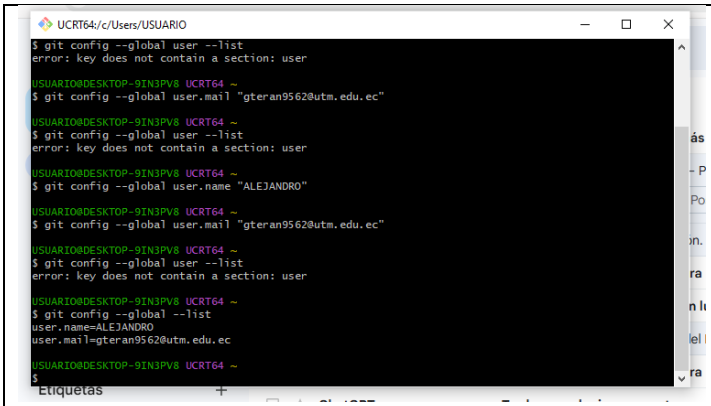


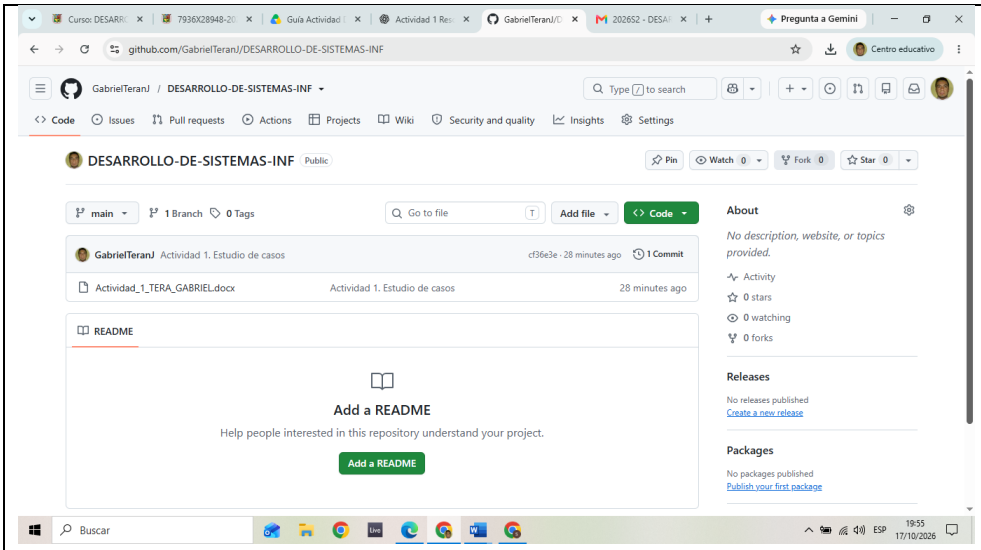
Diagrama de casos de uso para entender las interacciones del personal con la plataforma.



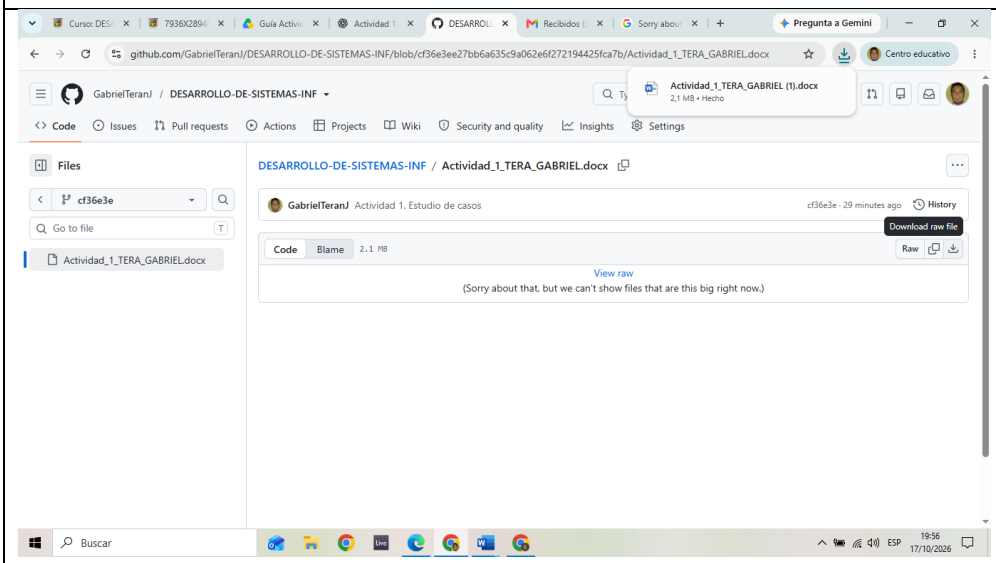
Evidencias



Ingreso de códigos que se deben ingresar por primera vez, nombre y correo



Una vez realizado los paso pertinentes, se encuentra subido el primer documento al git



El archivo se encuentra subido y para descargar

Link de lucidchart:

https://lucid.app/lucidchart/98f1d0db-760f-4928-ade4-93eed0d333e6/edit?viewport_loc=32%2C942%2C2140%2C1122%2CHWEp-vi-RSFO&invitationId=inv_649dc95d-cd52-4817-8c83-e05de2adfa7d

link de Github

https://github.com/GabrielTeranJ/DESARROLLO-DE-SISTEMAS-INF.git

Bibliografía

GitHub. (s. f.). GitHub Docs: About branches. https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-branches

Lucidchart.com